

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №2

По дисциплине «Логика и основы алгоритмизации в инженерных задачах»

На тему «Оценка времени выполнения программ»

Выполнил студент группы

24ВВВ2:

Тремасов К. И.

Приняли:

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Научиться оценивать времена выполнения программ языка Си и сопоставлять со сложностью программы.

Лабораторное задание

Задание 1:

- Вычислить порядок сложности программы (O-символику).
- Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.
- Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2:

- Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
- Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
- Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
- Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
- Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Описание метода решения задачи

Добавляем в указанную в методических материалах изменения, необходимые для оценки времени работы массива:

- Для умножения массивов добавим динамическое выделение памяти под массивы и их элементы;

- Используем функции библиотеки `time.h` для оценки времени выполнения алгоритма;
- Освобождаем память по окончании работы алгоритма

Порядок сложности программы

```
for (int i = 0; i < SIZE; i++) {  
    a[i] = (int*)malloc(SIZE * sizeof(int));  
    b[i] = (int*)malloc(SIZE * sizeof(int));  
    c[i] = (int*)malloc(SIZE * sizeof(int));  
}
```

Сложность $O(\text{SIZE})$

```
for (int i = 0; i < SIZE; i++) {  
    for (int j = 0; j < SIZE; j++) {  
        a[i][j] = rand() % 100 + 1;  
    }  
}
```

Сложность $O(\text{SIZE}^2)$

```
for (int i = 0; i < SIZE; i++) {  
    for (int j = 0; j < SIZE; j++) {  
        int elem_c = 0;  
        for (int r = 0; r < SIZE; r++) {  
            elem_c += a[i][r] * b[r][j];  
        }  
        c[i][j] = elem_c;  
    }  
}
```

Сложность $O(\text{SIZE}^3)$

```
for (int i = 0; i < SIZE; i++) {  
    free(a[i]);  
    free(b[i]);  
    free(c[i]);  
}
```

Сложность: $O(\text{SIZE})$

Суммируя все операции:

$O(\text{SIZE}) + O(\text{SIZE}^2) + O(\text{SIZE}^2) + O(\text{SIZE}^3) + O(\text{SIZE})$

Порядок сложности программы: $O(n^3)$

График зависимости времени выполнения программы от размера матриц

Таблица 1

Размер матриц	O-большое (количество операций)	Время выполнения (сек)
100	1 000 000	0.002
200	8 000 000	0.012
400	16000000	0.082
1000	1000000000	1.354
2000	8 000 000 000	20.226
4000	16 000 000 000	289.681
10000	1 000 000 000 000	2260.304

Зависимость времени выполнения программы от размера матриц

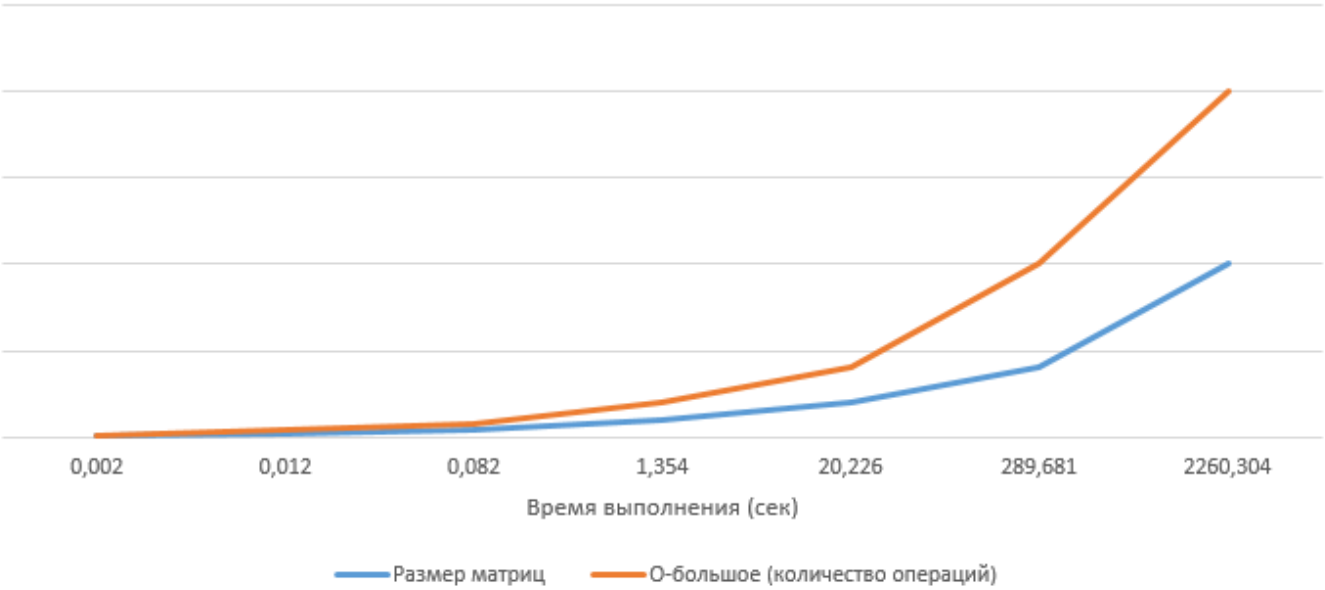


Рисунок 1

Время выполнения программы напрямую зависит от размера матриц: чем больше размер матриц, тем дольше будет выполняться программа. График значений O -большого повторяет форму графика размеров матриц, это показывает их прямую пропорциональность.

Оценка времени работы каждого из реализованных алгоритмов на различных видах массивов

Массив размером 50 000

Таблица 2

Алгоритм	Случайный массив	Возрастающий массив	Убывающий массив	Смешанный массив
shell	0,102 с	0,000 с	0,180 с	0,045 с
qs	0,003 с	0,001 с	0,001 с	0,001 с
qsort	0,004 с	0,003 с	0,003	0,003 с

Массив размером 500 000

Таблица 3

Алгоритм	Случайный массив	Возрастающий массив	Убывающий массив	Смешанный массив
shell	8,734 с	0,004 с	22,087 с	4,919 с
qs	0,029 с	0,006 с	0,008 с	0,007 с
qsort	0,039 с	0,030 с	0,041 с	0,038 с

По результатам тестирования можно наблюдать наглядно сильные и слабые стороны каждого алгоритма.

Алгоритм shell в среднем занимал больше всего времени на сортировку массивов данных. В малом и большом возрастающем массиве shell показал лучший результат, но в сортировке убывающего массива он сильно отставал от qs и qsort, а при увеличении размера массива разница увеличилась.

Сортировка shell в среднем оказалась наименее эффективной по потраченному времени.

Алгоритм qs оказался оптимальным. Он был быстрее в большинстве случаев сортировки массивов (кроме сортировки возрастающих массивов). Выполнение сортировки массива случайных чисел занимало больше всего времени, но все еще было меньше времени shell и qsort.

Алгоритм qsort хорошо показал себя в сортировке массивов меньшего размера, но стал сдавать позиции при увеличении массивов. Он ни разу не показал лучшее время выполнения. Несмотря на это, в больших массивах независимо от построения время выполнения алгоритма было почти неизменным.

Листинг программы

Файл main.c 1 задания

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 2000

int main(void)
{
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);

    clock_t start_all;
    float duration_all;
    start_all = clock();

    int** a = (int**)malloc(SIZE * sizeof(int*));
    int** b = (int**)malloc(SIZE * sizeof(int*));
    int** c = (int**)malloc(SIZE * sizeof(int*));
```

```

for (int i = 0; i < SIZE; i++) {
    a[i] = (int*)malloc(SIZE * sizeof(int));
    b[i] = (int*)malloc(SIZE * sizeof(int));
    c[i] = (int*)malloc(SIZE * sizeof(int));
}

srand(time(NULL));

for (int i = 0; i < SIZE; i++) {
    for (int j = 0; j < SIZE; j++) {
        a[i][j] = rand() % 100 + 1;
    }
}

for (int i = 0; i < SIZE; i++) {
    for (int j = 0; j < SIZE; j++) {
        b[i][j] = rand() % 100 + 1;
    }
}

clock_t start;
float duration;
start = clock();

for (int i = 0; i < SIZE; i++) {
    for (int j = 0; j < SIZE; j++) {
        int elem_c = 0;
        for (int r = 0; r < SIZE; r++) {
            elem_c += a[i][r] * b[r][j];
        }
        c[i][j] = elem_c;
    }
}

duration = (float)(clock() - start) / CLOCKS_PER_SEC;
duration_all = (float)(clock() - start_all) / CLOCKS_PER_SEC;

printf("Matrix multiplication execution time: %f seconds\n", duration);

```

```

printf("Total execution time: %f seconds\n", duration_all);

for (int i = 0; i < SIZE; i++) {
    free(a[i]);
    free(b[i]);
    free(c[i]);
}
free(a);
free(b);
free(c);
return 0;
}

```

Файл main.c 2 задания

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void shell(int* items, int count)
{
    int i, j, gap, k;
    int x;
    int a[5] = { 9, 5, 3, 2, 1 };

    for (k = 0; k < 5; k++) {
        gap = a[k];
        for (i = gap; i < count; ++i) {
            x = items[i];
            for (j = i - gap; (j >= 0) && (x < items[j]); j = j - gap)
                items[j + gap] = items[j];
            items[j + gap] = x;
        }
    }
}

void qs(int* items, int left, int right)

```

```

{
    int i, j;
    int x, y;

    i = left; j = right;
    x = items[(left + right) / 2];

    do {
        while ((items[i] < x) && (i < right)) i++;
        while ((x < items[j]) && (j > left)) j--;

        if (i <= j) {
            y = items[i];
            items[i] = items[j];
            items[j] = y;
            i++; j--;
        }
    } while (i <= j);

    if (left < j) qs(items, left, j);
    if (i < right) qs(items, i, right);
}

int compare(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

int* copy_array(const int* src, int count) {
    int* dest = (int*)malloc(count * sizeof(int));
    if (dest == NULL) {
        printf("Memory allocation error\n");
        exit(1);
    }
    for (int i = 0; i < count; i++) {
        dest[i] = src[i];
    }
    return dest;
}

```

```

void generate_random_array(int* arr, int count) {
    for (int i = 0; i < count; i++) {
        arr[i] = rand() % 1000;
    }
}

```

```

double measure_time(void (*sort_func)(int*, int), int* arr, int count) {
    clock_t start, end;
    start = clock();
    sort_func(arr, count);
    end = clock();
    return ((double)(end - start)) / CLOCKS_PER_SEC;
}

```

```

double measure_time_qs(void (*sort_func)(int*, int, int), int* arr, int
count) {
    clock_t start, end;
    start = clock();
    sort_func(arr, 0, count - 1);
    end = clock();
    return ((double)(end - start)) / CLOCKS_PER_SEC;
}

```

```

double measure_time_qsort(int* arr, int count) {
    clock_t start, end;
    start = clock();
    qsort(arr, count, sizeof(int), compare);
    end = clock();
    return ((double)(end - start)) / CLOCKS_PER_SEC;
}

```

```

int input_array_size() {
    int size;
    printf("Enter array size for testing (100 - 100000): ");
    if (scanf_s("%d", &size) != 1) {
        printf("Invalid input. Using default size 1000\n");
        return 1000;
    }
}

```

```

    }

    if (size < 100) {
        printf("Size too small, set to 100\n");
        return 100;
    }
    if (size > 100000) {
        printf("Size too large, set to 100000\n");
        return 100000;
    }
    return size;
}

int main() {
    srand(time(NULL));

    int count;
    char choice;

    do {
        printf("\n===== \n");
        printf("      SORTING ALGORITHMS BENCHMARK\n");
        printf("===== \n\n");

        count = input_array_size();
        printf("\n");

        int* random_array = (int*)malloc(count * sizeof(int));
        int* ascending_array = (int*)malloc(count * sizeof(int));
        int* descending_array = (int*)malloc(count * sizeof(int));
        int* mixed_array = (int*)malloc(count * sizeof(int));

        if (random_array == NULL || ascending_array == NULL ||
            descending_array == NULL || mixed_array == NULL) {
            printf("Memory allocation error\n");
            return 1;
        }
    }

```

```

generate_random_array(random_array, count);

for (int i = 0; i < count; i++) {
    ascending_array[i] = i;
}

for (int i = 0; i < count; i++) {
    descending_array[i] = count - i - 1;
}

int half = count / 2;
for (int i = 0; i < half; i++) {
    mixed_array[i] = i;
}
for (int i = half; i < count; i++) {
    mixed_array[i] = count - i - 1 + half;
}

printf("Testing with array size %d:\n\n", count);

printf("=== 1. Random Array ===\n");
int* arr = copy_array(random_array, count);
double time = measure_time(shell, arr, count);
printf("Shell Sort: %.6f seconds\n", time);
free(arr);

arr = copy_array(random_array, count);
time = measure_time_qs(qs, arr, count);
printf("Quick Sort: %.6f seconds\n", time);
free(arr);

arr = copy_array(random_array, count);
time = measure_time_qsort(arr, count);
printf("Standard qsort: %.6f seconds\n\n", time);
free(arr);

printf("=== 2. Ascending Array ===\n");
arr = copy_array(ascending_array, count);

```

```
time = measure_time(shell, arr, count);  
printf("Shell Sort: %.6f seconds\n", time);  
free(arr);
```

```
arr = copy_array(ascending_array, count);  
time = measure_time_qs(qs, arr, count);  
printf("Quick Sort: %.6f seconds\n", time);  
free(arr);
```

```
arr = copy_array(ascending_array, count);  
time = measure_time_qsort(arr, count);  
printf("Standard qsort: %.6f seconds\n\n", time);  
free(arr);
```

```
printf("=== 3. Descending Array ===\n");  
arr = copy_array(descending_array, count);  
time = measure_time(shell, arr, count);  
printf("Shell Sort: %.6f seconds\n", time);  
free(arr);
```

```
arr = copy_array(descending_array, count);  
time = measure_time_qs(qs, arr, count);  
printf("Quick Sort: %.6f seconds\n", time);  
free(arr);
```

```
arr = copy_array(descending_array, count);  
time = measure_time_qsort(arr, count);  
printf("Standard qsort: %.6f seconds\n\n", time);  
free(arr);
```

```
printf("=== 4. Mixed Array ===\n");  
arr = copy_array(mixed_array, count);  
time = measure_time(shell, arr, count);  
printf("Shell Sort: %.6f seconds\n", time);  
free(arr);
```

```
arr = copy_array(mixed_array, count);  
time = measure_time_qs(qs, arr, count);
```

```

printf("Quick Sort: %.6f seconds\n", time);
free(arr);

arr = copy_array(mixed_array, count);
time = measure_time_qsort(arr, count);
printf("Standard qsort: %.6f seconds\n\n", time);
free(arr);

free(random_array);
free(ascending_array);
free(descending_array);
free(mixed_array);

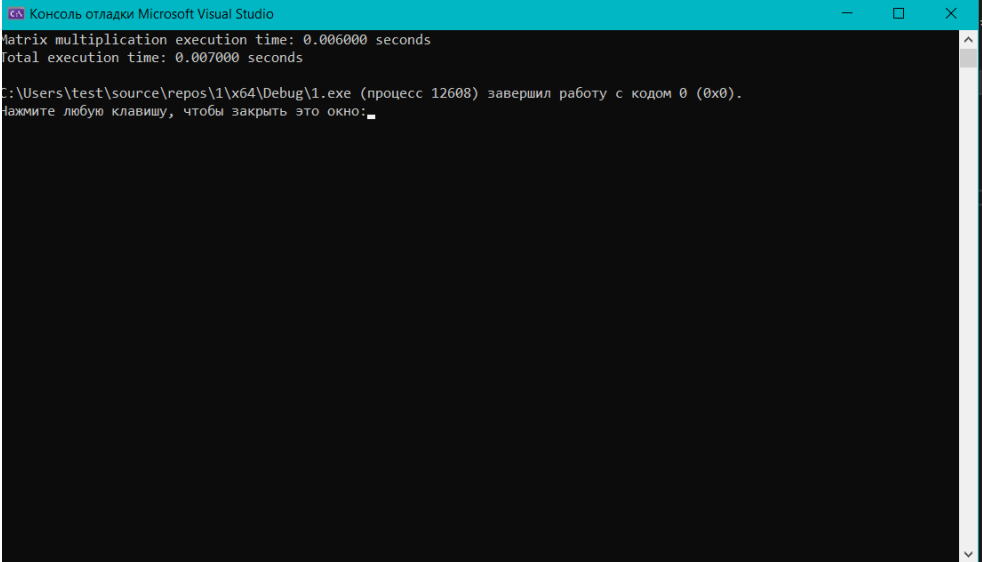
printf("Test with different array size? (y/n): ");
if (scanf_s(" %c", &choice, 1) != 1) {
    choice = 'n';
}

} while (choice == 'y' || choice == 'Y');

printf("Program finished.\n");
return 0;
}

```

Результат работы программы



The screenshot shows the 'Консоль отладки Microsoft Visual Studio' (Visual Studio Debug Console) window. It displays the following text:

```

Matrix multiplication execution time: 0.006000 seconds
Total execution time: 0.007000 seconds

C:\Users\test\source\repos\1\Debug\1.exe (процесс 12608) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:

```

The console window has a standard Windows title bar with minimize, maximize, and close buttons. The background is black with white text.

```
Консоль отладки Microsoft Visual Studio
Matrix multiplication execution time: 0.020000 seconds
Total execution time: 0.023000 seconds

C:\Users\test\source\repos\1\x64\Debug\1.exe (процесс 7240) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

```
Консоль отладки Microsoft Visual Studio
Matrix multiplication execution time: 0.198000 seconds
Total execution time: 0.213000 seconds

C:\Users\test\source\repos\1\x64\Debug\1.exe (процесс 12732) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" -> "Параметры" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

```
Консоль отладки Microsoft Visual Studio
Matrix multiplication execution time: 4.997000 seconds
Total execution time: 5.083000 seconds

C:\Users\test\source\repos\1\x64\Debug\1.exe (процесс 15872) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

```
Консоль отладки Microsoft Visual Studio
Matrix multiplication execution time: 15.892000 seconds
Total execution time: 16.051001 seconds

C:\Users\ardor\source\repos\Ла62\x64\Debug\Ла62.exe (процесс 16060) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

```
Консоль отладки Microsoft Visual Studio
Matrix multiplication execution time: 265.740997 seconds
Total execution time: 266.377991 seconds

C:\Users\ardor\source\repos\Ла62\x64\Debug\Ла62.exe (процесс 4484) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

```
C:\Users\test\source\repos\2\x64\Debug\2.exe

=====
      SORTING ALGORITHMS BENCHMARK
=====

Enter array size for testing (100 - 100000): 50000
Testing with array size 50000:

=== 1. Random Array ===
Shell Sort: 0.154000 seconds
Quick Sort: 0.003000 seconds
Standard qsort: 0.006000 seconds

=== 2. Ascending Array ===
Shell Sort: 0.001000 seconds
Quick Sort: 0.001000 seconds
Standard qsort: 0.004000 seconds

=== 3. Descending Array ===
Shell Sort: 0.312000 seconds
Quick Sort: 0.001000 seconds
Standard qsort: 0.005000 seconds

=== 4. Mixed Array ===
Shell Sort: 0.076000 seconds
Quick Sort: 0.002000 seconds
Standard qsort: 0.004000 seconds

Test with different array size? (y/n):
```

```
C:\Users\ardor\source\repos\lab2zad2\Debug\lab2zad2.exe
Enter array size for testing (100 - 1000000): 500000
Warning: Large array size (500000 elements)
Testing may take significant time...
Generating test arrays...
Testing with array size 500000:

=== 1. Random Array ===
Shell Sort: 8.855000 seconds
Quick Sort: 0.029000 seconds
Standard qsort: 0.039000 seconds

=== 2. Ascending Array ===
Shell Sort: 0.004000 seconds
Quick Sort: 0.006000 seconds
Standard qsort: 0.030000 seconds

=== 3. Descending Array ===
Shell Sort: 18.760000 seconds
Quick Sort: 0.006000 seconds
Standard qsort: 0.031000 seconds

=== 4. Mixed Array ===
Shell Sort: 4.191000 seconds
Quick Sort: 0.006000 seconds
Standard qsort: 0.031000 seconds

Test with different array size? (y/n):
```

Вывод: Из оценок времени работы программы в первом задании был сделан вывод, что с повышением размера перемножаемых матриц увеличивается количество элементарных операций, что приводит к увеличению количества времени в секундах, которое тратится на выполнение программы.

Во втором задании оценка времени работы каждого из реализованных алгоритмов (shell, qs, qsort) на различных видах массивов показала, что в большинстве случаев рациональнее использовать qs, т. к. по затратам времени в среднем он показал наилучший результат. Алгоритм shell выделился при сортировке возрастающего массива, поэтому qs не являлся самым оптимальным всегда. Алгоритм qsort показал наихудшее время выполнения, т. к. он является методом стандартной библиотеки C, предназначенным для корректной работы с массивами любых типов данных, qs и shell обгоняли qsort из-за более узкой направленности.

Во время выполнения лабораторной работы научились оценивать время выполнения программ языка Си и сопоставлять со сложностью программ.